

Inteligencia Artificial

Trabajo Práctico N° 2: Diseño e Implementación de Agentes Inteligentes basados en LLMs

Fecha de entrega: **29/06/2026**

Objetivo

El objetivo de este trabajo práctico es diseñar, implementar y evaluar un agente inteligente autónomo que utilice modelos de lenguaje (LLMs, como GPT, Claude, Mistral, y otros) como parte central de su arquitectura de razonamiento. El agente deberá resolver un problema concreto o asistir en una tarea específica dentro de un entorno definido por el grupo.

Se espera puedas aplicar conceptos de sistemas basados en agentes, razonamiento, planificación, percepción y acción, integrando además técnicas modernas de inteligencia artificial generativa como RAG (Retrieval-Augmented Generation), herramientas (Tools), y memoria contextual.

Descripción

Un agente inteligente es un sistema capaz de percibir su entorno, razonar, tomar decisiones y actuar de forma autónoma para alcanzar un objetivo. En este TP, deberás construir un agente de software que utilice un LLM como componente fundamental para, entre otras cosas, realizar razonamientos, interpretar lenguaje natural y/o ejecutar acciones.

Cada grupo deberá proponer un caso de uso concreto en el cual su agente pueda demostrar capacidades útiles. Algunos ejemplos posibles incluyen:

- Un asistente virtual para atención al cliente
- Un chatbot que responda dudas legales o académicas
- Un agente que ayude a planificar viajes o eventos
- Un tutor educativo que explique temas de una materia
- Un agente que juegue un juego de texto de forma autónoma
- Un asistente en línea de una app web (por ejemplo, para onboarding)
- Un sistema que interactúe con herramientas externas (API, navegador, base de datos)

- Un agente que opere un navegador web de forma autónoma (computer use / browser-use) para tareas como reservas, búsquedas estructuradas o scraping inteligente
- Un agente que escriba y ejecute código para resolver un problema (análisis de datos, generación de scripts, automatización de tareas)
- Un agente de investigación multi-step que produzca un informe sobre un tema (búsqueda + lectura + síntesis + citación de fuentes)
- Un sistema multi-agente con roles diferenciados (por ejemplo: planner + executor + reviewer) trabajando sobre una misma tarea

El agente debe incorporar además técnicas como:

- RAG: búsqueda de información en fuentes externas (por ejemplo, bases de datos, Wikipedia, PDFs, páginas web, etc.)
- Tools: acciones que el agente puede ejecutar como parte de su razonamiento (llamar APIs, programar recordatorios, ejecutar cálculos, buscar en internet)
- Memoria: mantener contexto de interacciones pasadas para decisiones futuras

Mínimo obligatorio: el agente debe incorporar Tools (al menos 2-3 funcionales) y Memoria conversacional. Adicionalmente, debe incluir al menos uno de los siguientes diferenciadores: RAG sobre fuente externa, módulo de planificación multi-step, o arquitectura multi-agente.

Frameworks recomendados (ejemplos, no obligatorios): LangGraph, Pydantic AI, CrewAI, LlamaIndex, Agno, Pi, OpenCode, Hermes. También se puede construir el agente desde cero usando directamente los SDK de los proveedores (Anthropic, OpenAI, Mistral, etc.) si el grupo prefiere control total sobre el loop del agente.

Evaluación del agente: el grupo debe definir cómo va a probar que el agente funciona. Se sugiere construir un pequeño conjunto de casos de prueba (entre 5 y 15) que cubran: (a) escenarios principales (happy path), (b) casos límite o ambiguos, y (c) entradas adversariales o fuera de dominio. Para cada caso, definir el comportamiento esperado y registrar el comportamiento observado. Es válido usar LLM-as-judge para criterios subjetivos (calidad de respuesta), pero las invocaciones a tools deben validarse de forma determinística.

Observabilidad: el agente debe permitir inspeccionar qué hace en cada paso. Como mínimo, debe haber logging de las llamadas al LLM (prompts y respuestas) y de las invocaciones a tools (entrada y salida). Se sugiere usar herramientas como Arize Phoenix, Langfuse, BrainTrust, LangSmith o Weave para trazas estructuradas, aunque también es válido implementar un logging propio.

El entorno de ejecución deberá contar con una interfaz de usuario interactiva básica. Puede ser una interfaz web o de escritorio. Por ejemplo, una aplicación en javascript con React, Next.js, Vue.js, etc., o en Python con Streamlit, Gradio, etc., o incluso un entorno gráfico de simulación o juego.

El objetivo es que se pueda interactuar con el agente de forma clara y demostrable, permitiendo evaluar su comportamiento ante distintos escenarios. Esto es muy importante para el coloquio.

Etapas y Entregas

En este trabajo práctico, se deja abierta la elección del dominio donde el agente se desenvolverá, y se evaluará con entregas parciales:

1° entrega (8/6) Definición conceptual:

- Descripción general del problema
- Definición del **objetivo** del agente
- Identificación de **percepciones** y **acciones** (o Tools).
- Especificar el **ambiente** en donde el agente se desenvuelve.
- Definición de la arquitectura del agente: módulos, tecnologías y componentes.

2° entrega (22/06) Avances en la implementación (ver aclaración más abajo):**

- Implementación base del agente,
- Flujo de interacción con el LLM
- Implementación del módulo de contexto y manejo del ambiente,
- Conexiones externas necesarias (APIs, base de datos, etc.),
- Workflow.
- Integración de herramientas (tools) y fuentes externas (RAG),
- Loop / estrategia de razonamiento del agente
- Logging básico de las llamadas al LLM y tools

3° entrega (29/06) Final:

- Implementación completa del agente,
- Defensa con coloquio.

**** Aclaración sobre el alcance de la segunda entrega:** en cada entrega, todos los puntos listados deben estar **conceptualmente definidos y documentados** (decisiones de diseño tomadas, justificadas y registradas en el informe). En paralelo, se espera avance en la **implementación mínima funcional** de los componentes correspondientes a esa entrega: no se exige que estén pulidos ni completos, pero sí que exista un esqueleto ejecutable que demuestre que las decisiones conceptuales son viables. La idea es evitar que la implementación total se concentre en la última semana.

Criterios de evaluación

Cada criterio se evalúa sobre una escala de cuatro niveles: insuficiente, aceptable, bueno, excelente. La aprobación del TP requiere alcanzar al menos el nivel “aceptable” en todos los criterios y “bueno” en al menos la mitad.

- Originalidad y claridad del caso de uso
- Correcta aplicación de conceptos de agentes y LLMs
- Diseño modular, arquitectura clara
- Capacidad del agente de razonar y actuar coherentemente
- Uso justificado de herramientas, memoria, RAG o planificación
- Calidad del código y documentación
- Evaluación realizada por el grupo: definición de un conjunto de casos de prueba (escenarios principales, casos límite y entradas adversariales), análisis crítico de fortalezas y debilidades del agente, y reporte honesto de los modos de falla detectados
- Observabilidad y trazabilidad: posibilidad de inspeccionar las decisiones del agente paso a paso (logs, trazas o dashboards)
- Defensa oral con demostración funcional en vivo del agente y claridad conceptual

Formato de entrega

Cada grupo deberá entregar:

- **Código fuente en GitHub:** repositorio público con todo el código del agente. Debe incluir un README con instrucciones claras de instalación y ejecución, las dependencias declaradas (requirements.txt, package.json, etc.) y, si se usan API keys, un .env.example.
- **Informe técnico:** documento en formato PDF siguiendo la estructura del Anexo I.
- **Defensa presencial:** en la fecha de la entrega final, cada grupo presentará el agente de forma presencial, con demostración en vivo del comportamiento del sistema ante distintos escenarios.

Documentación a presentar:

Se deberá elaborar un informe técnico con el formato propuesto en el **Anexo I**.

ANEXO I: Formato del informe del TP

Nombre del TP

Nro. de Grupo

Nombre y Apellido integrante1 - e-mail

Nombre y Apellido integrante2 - e-mail

Nombre y Apellido integrante3 - e-mail

Resumen. Acá se escribe un pequeño resumen del trabajo que se presenta. Por ejemplo, la aplicación de IA que se va a hacer, el problema concreto que se va a resolver, si fue o no resuelto y cómo, y los resultados que se presentan. Todo en pocas palabras (entre 70 y 150 palabras).

1 Introducción

En esta sección se introduce el área de aplicación en la que se va a trabajar, se explica el problema que se va a resolver. Se puede usar una figura o esquema para explicar mejor lo que se quiere hacer en el trabajo.

Se puede mostrar un gráfico con los datos que se están usando. En ese caso se diría p.e. "los datos usados para el entrenamiento se pueden ver en la figura 1, ...". Esto quiere decir que "...". Esta forma de nombrar los gráficos se mantiene para todo el informe, es decir, se usará este formato cada vez que se presente una figura.

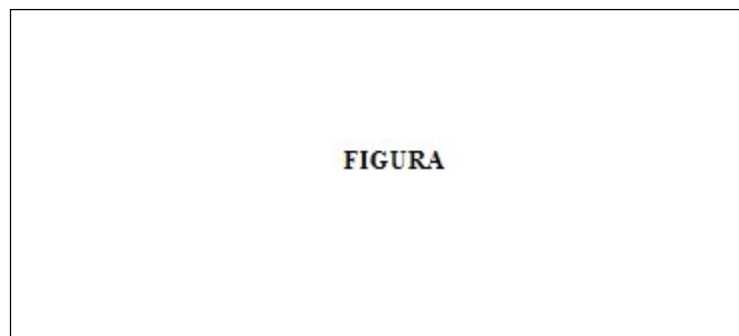


Figura 1. Explicación de lo que se ve en la figura.

Si los datos o alguna otra información a lo largo del trabajo se quiere presentar en forma de tabla, acá se muestra un formato posible como ejemplo.

XX	XXXX	
Col 1	Col 2	Col 3
xxx	xx.xx	xx.xx
xxx	xx.xx	xx.xx
xxx	xx.xx	xx.xx
xxx	xx.xx	xx.xx

Tabla 1. Explicación de lo que se ve en la tabla.

Generalmente, al final de la introducción se describe cómo sigue el informe, es decir, se explica que hay en cada sección siguiente. Por ejemplo: “en la sección 2 se explica En la sección 3 se muestra Finalmente en la sección xx ...”.

2 Solución

En esta parte se debería explicar la solución conceptual del problema describiendo los componentes propios de un agente basado en LLM: (a) arquitectura general del agente con diagrama de componentes y flujo de control; (b) system prompt y estrategia de prompting utilizada; (c) tools / function calling, indicando schema, validaciones aplicadas y manejo de errores; (d) loop del agente (por ejemplo ReAct, Plan-and-Execute, Reflection u otro) justificando la elección; (e) memoria: tipo (short-term en context window, long-term en vector store, etc.) y política de actualización; (f) RAG, si aplica: estrategia de chunking, modelo de embeddings, retriever y posibles re-rankers; (g) guardrails y validación: límites de iteración, validación de inputs/outputs, manejo de alucinaciones; (h) observabilidad: cómo se registran trazas, logs y costos de la ejecución. Si se aplicó alguna metodología para resolver el problema, explicarla.

Justificar la solución y las elecciones hechas.

Si se va a hacer alguna comparación, explicar entre qué cosa y qué cosa, y por qué se comparan. Mostrar por ejemplo algún gráfico con el modelo del problema resuelto.

Si se quiere escribir alguna ecuación, la forma de hacerlo se muestra acá abajo. Se coloca la ecuación en el texto (*es un objeto equation en word*) y a la derecha se pone un número para identificarla, que aumenta secuencialmente a medida que se agregan más ecuaciones al informe.

$$y = x \quad (1)$$

3 Resultados

En esta sección se deberían mostrar las pruebas que se han hecho para verificar que la solución al problema propuesto funciona y explicar los resultados obtenidos.

Se deben mostrar trazas representativas de ejecución del agente: invocaciones al LLM (con prompts y respuestas), llamadas a tools (con inputs y outputs), decisiones tomadas en el loop del agente, y la respuesta final entregada al usuario. Es deseable incluir también métricas agregadas como cantidad de pasos por tarea, tokens consumidos y costo estimado.

Se pueden mostrar gráficos o tablas con los resultados obtenidos de las ejecuciones, con los errores obtenidos, etc.

Si se trató de resolver un problema, hay que mostrar cómo el agente lo resolvió (o no), o si se buscaba una respuesta a una pregunta, cuál es la respuesta que brinda el agente propuesto.

4 Conclusiones

En esta sección se deben obtener conclusiones del trabajo presentado.

Que conclusión se puede sacar luego de haber aplicado una técnica de IA para resolver un problema. Si el modelo propuesto para resolver el problema es bueno o no, por qué, ventajas, desventajas, puntos positivos, puntos negativos, etc...

ACLARACION: este documento pretende ser de base en cuanto al FORMATO del trabajo práctico, es decir, el tipo de letra, tamaño, como mostrar figuras y tablas, etc., para uniformar las presentaciones de los distintos grupos. Los nombres de las secciones son sugerencias, no etiquetas obligatorias. Cada grupo elegirá la cantidad y nombres de secciones y el tipo y cantidad de información que agregará al informe, según el problema que haya (o no) resuelto.

Referencias (aclaración: si se consultaron libros, o papers, o se bajaron datos de internet, etc., se deben colocar las referencias en esta sección)

1. Apellido, Nombre: Nombre LIBRO. Editorial (año)
2. Apellido, Nombre: Nombre PAPER. Nombre REVISTA o CONGRESO, volumen, numero, nro. de paginas (desde-hasta), (año)

EJEMPLOS

1. Martin del Brio, B., Sanz Molina, A.: Redes Neuronales y sistemas difusos. Ed. Alfaomega (2002)
2. Meireles, M.R.G., Almeida, P.E.M., Simoes, M.G.: A comprehensive review for the industrial applicability of Artificial Neural Networks. IEEE Transactions on Industrial Electronics, vol. 5, no. 3, pp. 585-601 (2003)
3. <http://www.iee.org>

EJEMPLOS ADICIONALES (referencias recomendadas para el TP de agentes basados en LLMs):

4. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach. 4th edition, Pearson (2020) – capítulos sobre agentes inteligentes
5. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing Reasoning and Acting in Language Models. ICLR (2023). <https://arxiv.org/abs/2210.03629>
6. Lewis, P., Perez, E., Piktus, A., et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS (2020). <https://arxiv.org/abs/2005.11401>
7. Shinn, N., Cassano, F., Berman, E., et al.: Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS (2023). <https://arxiv.org/abs/2303.11366>
8. Schick, T., Dwivedi-Yu, J., Dessí, R., et al.: Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS (2023). <https://arxiv.org/abs/2302.04761>
9. Wang, L., Ma, C., Feng, X., et al.: A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science (2024). <https://arxiv.org/abs/2308.11432>
10. Anthropic: Building Effective Agents (2024). <https://www.anthropic.com/research/building-effective-agents>
11. Model Context Protocol (MCP) Specification. <https://modelcontextprotocol.io>